From Code to EM Signals: A Generative Approach to Side Channel Analysis-based Anomaly Detection

Kurt A. Vedros
kvedros@uidaho.edu
Computer Science Department
University of Idaho
Idaho Falls, Idaho, USA

Constantinos Kolias
kolias@uidaho.edu
Computer Science Department
University of Idaho
Idaho Falls, Idaho, USA

Daniel Barbará
dbarbara@gmu.edu
George Mason University
Fairfax County, Virginia, USA

Robert C. Ivans
robert.ivans@inl.gov
Idaho National Labs
Idaho Falls, Idaho, USA

Abstract

Today, it is possible to perform external anomaly detection by analyzing the involuntary EM emanations of digital device components. However, one of the most important challenges of these methods is the manual collection of EM signals for fingerprinting. Indeed, this procedure must be conducted by a human expert and requires high precision. In this work, we introduce a framework that alleviates this requirement by relying on synthetic EM signals that have been generated from assembly code. The signals are produced with the use of a Generative Adversarial Network (GAN) model. Experimentally, we identify that the synthetic EM signals are extremely similar to the real and thus, can be used for training anomaly detection models effectively. Through experimental assessments, we prove that the anomaly detection models are capable of recognizing even minute alterations to the code with high accuracy.

CCS Concepts: **• Security and privacy → Side-channel analysis**; **Intrusion/anomaly detection and malware mitigation**; **• Computer systems organization → Generative Artificial Networks**.

Keywords: Side Channel Analysis, Anomaly Detection, Generative Artificial Networks

ACM Reference Format:
Kurt A. Vedros, Constantinos Kolias, Daniel Barbará, and Robert C. Ivans. 2024. From Code to EM Signals: A Generative Approach to Side Channel Analysis-based Anomaly Detection. In *The 19th International Conference on Availability, Reliability and Security (ARES 2024), July 30-August 2, 2024, Vienna, Austria.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3664476.3664520

ARES 2024, July 30-August 2, 2024, Vienna, Austria

ACM ISBN 979-8-4007-1718-5/24/07
https://doi.org/10.1145/3664476.3664520

1 Introduction

In the context of hardware security, *side-channel analysis* traditionally, has been employed for adversarial tasks such as the cracking of cryptographic keys [8]. These methodologies rely mainly on the analysis of power consumption patterns or the involuntary electromagnetic (EM) emanations of digital device components. Recently, researchers have proven that the same set of principles can be applied for *defensive* purposes [5], [10], [20]. For example, one of the most important applications of side-channel analysis is to provide external *anomaly detection* remotely, without burdening the target device. Among the different modalities that can be used for the purpose e.g., power consumption, sound, thermal, the use of EM signals is becoming increasingly popular because the EM spectrum offers *higher bandwidth*, and supports *higher sampling rates* [14]. What is more, unlike the monitoring of power consumption, the collection of EM signals can be performed from a *distance*, completely disengaged from the target device [13]. Interestingly, recent research in the field has highlighted that it is possible to detect malicious modification of instructions with great precision, and extremely high resolution i.e., down to a few assembly instructions [12], [16].

Despite its benefits, the application of EM-based anomaly detection in real life appears to be stagnant. One of the most important challenges revolves around the process of collecting EM signals for fingerprinting. Ideally, this phase should be short, simple, and done only sporadically. However, in reality, the following practical challenges make this phase extremely costly.

1. The fingerprinting *process is manual*, requiring high precision and possibly the involvement of a human expert.
2. Most programs are comprised of *myriad alternative execution paths*, each of which needs to be fingerprinted.

3. With some programs, it is not straightforward to *trigger certain, rarely executed paths*.
4. It is common for software to go through *frequent update cycles*, which raises the need for fingerprinting from scratch.

All these factors contribute to analogous defense tools being deemed expensive, non-scalable, and impractical [18].

Inspired by existing text-to-speech [9] methods or ones that convert sketches to photorealistic images [11], we introduce a methodology for translating *code to EM signals* that is based on Generative Adversarial Networks (GAN). This mechanism lies in the epicenter of a comprehensive framework that aims to train highly effective anomaly detection models by relying mainly on artificially generated EM signals. By adopting this framework, we minimize the cost of the data-gathering phase which is the bottleneck of the fingerprinting phase. Thus, the proposed methods drastically reduce the cost of EM-based anomaly detection and contribute to the development of realistic and deployable defense tools in real-life settings.

2 Technical Background

EM-based anomaly detection capitalizes on the relationship between instructions executed at the machine level and the EM signals emitted involuntarily as the natural outcome of this process. Every time a new instruction is executed, slightly different amounts of current are drawn by the CPU. This results in the formation of EM fields, and therefore, the emanation of EM signals. To an external observer, these signals can act as indicators of the type of instructions executed by the CPU at any given time. The side-channel information is significant enough to perform anomaly detection to determine if the device has been compromised by malware [7]. However, this process necessitates the modeling of the normal operational state (at least). Due to environmental noise and random events occurring at the micro-architectural level, signals corresponding to the same event are not identical. For this reason, training is usually performed by capturing a large volume of signals from the device in a state known to be free of attacks. This is known as *fingerprinting* the device. EM-based fingerprinting is mainly a human-expert-centric process, comprising delicate tasks such as the correct positioning of probes, deciding the optimal recording parameters like the sampling rate, and synchronizing EM signals, among others. The complexity of the practical aspects of this process, in turn, renders the entire EM fingerprinting stage an extremely time-consuming, error-prone, and costly process.

This paper attempts to replace the human-centric approach of fingerprinting with one in which the needed signals are generated by AI methods drawn from the emerging area of Generative Modeling (GM).

GM is a branch of Deep Learning (DL) that revolves around training models to produce new data that is similar to a given dataset. In this context, this dataset is a representative sample of EM signals. We want to generate *synthetic* signals that follow the unknown (and unknowable) probability density distribution of real EM signals. We choose, in this paper, to focus on a class of GMs known as GANs.

A GAN is a DL architecture composed of two models: a *generator* and a *discriminator*, which perform a competition or *battle* against each other (hence the name adversarial). The generator tries to convert random noise into observations that look as if they have been sampled from the original dataset, and the discriminator tries to predict whether an observation comes from the original dataset or is one of the generator's forgeries. This process converges into a state where the generator learns to make perfect (yet, diverse) samples of the dataset, while the discriminator is rendered incapable of distinguishing between the real and the synthetic objects. This status of *equilibrium* can be proven to be a case of the Nash equilibrium [2].

2.1 WGANs

The architecture for Wasserstein GANs (WGANs) [1] was first introduced as a solution to improve the *stability* of the learning process. The issues of convergence in the original formulation of GANs come from two sources. First, the loss function, which can be proven to correspond to the Jensen-Shannon Divergence between the density of the generated data and the real density of the data (which becomes 0 when the equilibrium is reached) is not a very good discerning function of how close the model is to reaching equilibrium (in fact, it can only tell us whether we have reached it or not). Secondly, both densities are often located in small manifolds and it becomes unlikely that we can make them overlap by sampling. By applying *Wasserstein distance* (Earth-Moving distance) to the traditional loss function, the discriminator becomes a *critic*, giving a value to each instance instead of classifying between real and fake. Consequently, the value output provided improves feedback when training both the generator and discriminator models. Furthermore, the use of Wasserstein loss reduces common issues that traditional GANs suffer from. Later, WGANs were further improved by incorporating the principles of gradient penalty (WGANs-GP) [3]. The WGANs-GP loss function is provided in equation 1 below:

$$\min_G \max_D \underbrace{\mathop{\mathbb{E}}_{G(z)\sim\mathbb{P}_g}[D(G(z))] - \mathop{\mathbb{E}}_{x\sim\mathbb{P}_r}[D(x)]}_{Original\ W\_Loss} + \underbrace{\lambda \mathop{\mathbb{E}}_{\hat{x}\sim\mathbb{P}_{\hat{x}}}[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2]}_{Gradient\ Penalty} \quad (1)$$

where $\mathbb{P}_r$ is the data distribution of real data, $\mathbb{P}_g$ is the model distribution defined by $G(z)$, $D(x)$ is the discriminator's output for a real instance, $G(z)$ is the generator's output when given noise z and $D(G(z))$ is the discriminator's output for a fake instance, λ is the penalty coefficient and $\mathbb{P}_{\hat{x}}$ is the distribution sampling uniformly along straight lines between pairs of points sampled from $\mathbb{P}_r$ and $\mathbb{P}_g$. Figure 1 illustrates the workflow to obtain the Wassterstein loss adjusted by the gradient penalty.

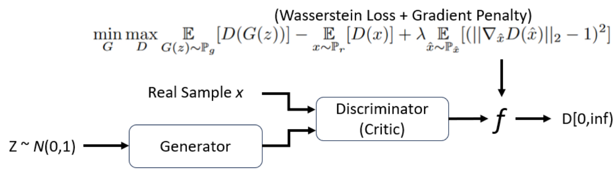

Figure 1. WGAN-GP workflow.

2.2 CGANs

Conditional GANs (CGANs) convert traditional GANs into a conditional-based model by providing auxiliary information (i.e., class labels) y to the generator and discriminator. In the generator's case, the input contains latent dimension noise z combined with y. It should be noted that the adversarial training framework allows for considerable flexibility on how the hidden representation is composed, however, the organization must remain the same throughout, e.g., y always be appended to z at the end. In the discriminator's case, the observations x and y are combined and presented as inputs to the discriminative function. Furthermore, the loss function of the traditional GANs model is updated to include the additional information of y as shown in equation 2. The typical workflow of the CGAN model is provided in Figure 2.

$$Loss = \min_G \max_D E_x[\log D(x\|y)] + E_z[\log 1 - D(G(z\|y))] \quad (2)$$

The two GAN models above work well in tandem. CGANs' conditional-based classes inform the generator and discriminator. Thus, the output is derived from the provided label.

2.3 ResGANs

ResGANs [19] incorporate a commonly used artificial Neural Network (NN) model known as Residual Network or ResNet [4] into the generator inside the GAN model. ResNet was originally designed for image recognition to address the degradation problem with accuracy getting saturated with larger network depth. To overcome this challenge, ResNet applies the concept of *Residual Learning* to a deep NN framework. *Residual Learning* is the process of using sections of NN layers and activation functions to reapply the identity mapping of the input after a few stacked layers. These sections are known as *Residual Blocks* and are often stacked

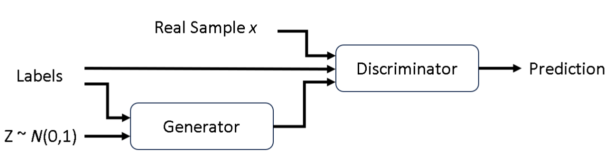

Figure 2. CGANs workflow.

multiple times in common ResNet variations. The operation of the residual blocks can be represented with equation 3.

$$output = F(x) + x \quad (3)$$

where x is the input to the residual block, $F(x)$ is a Convolutional NN consisting of several convolutional blocks.

The process of *Residual Learning* re-enforces prior input knowledge and avoids issues of vanishing/exploding gradients. The overall structure of the generator in ResGANs is provided in Figure 3.

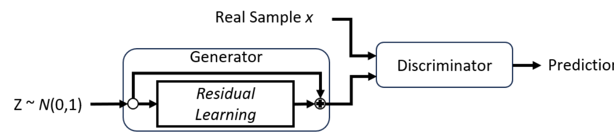

Figure 3. ResGANs workflow.

2.4 Tokenizer

Tokenizer is a simple vectorization tool, frequently used with textual data. Overall, the tokenizer converts common words or sentences into unique number representations that indicate the index in a collective dictionary. The *tokenizer dictionary* contains one-to-one references between words/sentences and index numbers.

2.5 Autoencoders

Autoencoders (AE) are a NN approach to representation learning [6]. The functionality of AE is based on dimensionality reduction, in which the primary hidden structures in a given dataset are identified and represented in a compressed form. This is achieved by two subparts: an encoding function that slowly reduces the input dimensionality $h = f(x)$ and a decoding function $y = g(x)$ that reconstructs the input given h. In essence, the AE learns the identity function $g(f(x)) = \hat{x}$, creating a near-perfect copy of the input. However, this copy $\hat{x}$ generally is created with a certain amount of difference, commonly called the *reconstruction error*.

3 Assumptions & Threat Model

The following assumptions are made about the capabilities of the attacker. We assume that the attacker has managed to acquire access to the source or binary control code of a system (typically an embedded device). Moreover, through the analysis of the given program, they have identified a given vulnerability that they have managed to exploit e.g., through a buffer overflow attack to inject their malicious code. Furthermore, while not frequently seen in real-life situations, it is assumed that the attacker can achieve meaningful malicious activity with minimal alterations to the base code i.e., changing, deleting, or adding one instruction. Additionally, the attacker attempts to conceal their activity by limiting their alterations to avoid detection through side-channel

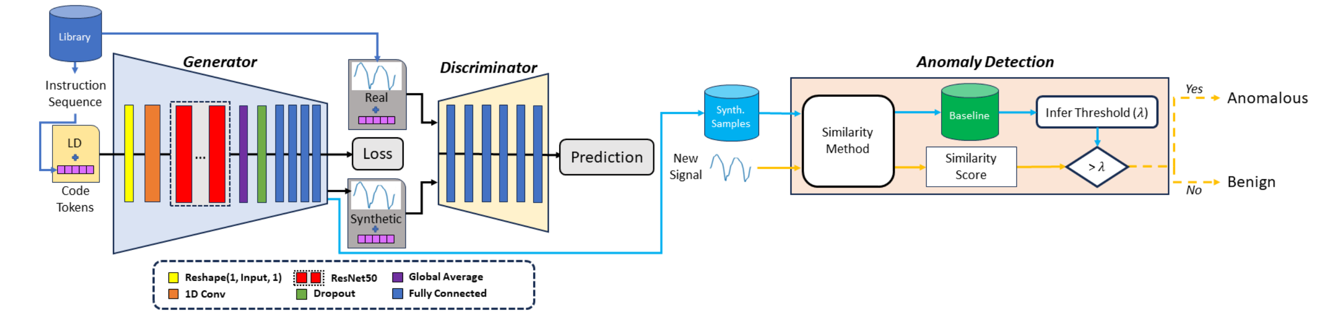

Figure 4. Proposed framework.

analysis. Finally, we assume that the attack will be on a device in an environment indicative of its normal use and not in a controlled lab environment. As such, environmental noise is expected to negatively impact the process.

4 Proposed Framework

The proposed framework relies on a novel approach to converting assembly (ASM) code to EM signals for anomaly detection purposes, by relying on principles of GM. The general workflow of the proposed framework is given in Figure 4.

4.1 Library Creation

A library containing the pairing between *basic building blocks* and EM samples produced by the said blocks is created a priori. Once obtained, the library is utilized to train the GM. The reader should note that the library only needs to be completed once for a given device as future GMs can be trained using that same library. Ideally, the *basic building blocks* would correspond to every individual ASM instruction. Consequently, the library would be created using a naive approach [15] where the analyst harvests signals corresponding to single instructions obtained from dummy programs where the given instruction is surrounded by *nops*. However, such a library would not account for the active device state during the execution of a given instruction. For example, a given instruction say, an *ldi* will have different characteristics e.g., amplitude if observed after two different sequences of instructions. This is further illustrated in Figure 5, as the *ldi* in Subfigure A is produced by said naive approach is dissimilar to the *ldis* in Subfigure B.

4.1.1 Accounting for Device State: The device state refers to all internal status, conditions, the electrical charge of capacitors, etc that currently characterize the device. The current device state can drastically affect the morphology of the EM signal produced when executing instructions. For example, this effect can be seen with instructions that require bit flips. Bit flips occur when the values of flags, registers, etc change. To change a bit, a certain amount of electricity is required. As such, changing a value in a way that requires

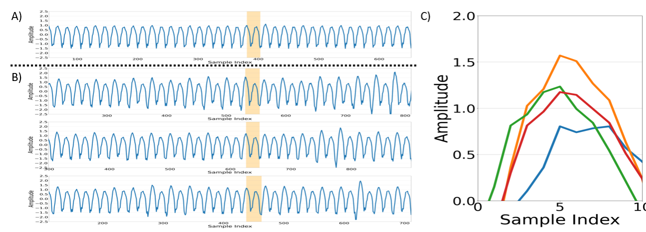

Figure 5. A. *Ldi* seen surrounded by *nops*. B. *Ldi* seen in indifferent programs. C. *Ldi* peak amplitude seen in A and B.

more bit flips would result in a higher EM amplitude spike than those that require less, even for the same ASM instruction. The effect of the active device state during runtime can be seen in Subfigure B of Figure 5, where the amplitude corresponding to the same instruction is different in separate programs. This implies that one needs to take into account all possible device states when generating synthetic EM signals. However, this is infeasible in practice as often, a myriad of alternative states exist. Instead, to obtain a close approximation of the EM produced from a given state, the framework takes into account sequences of instructions.

Some of the EM phenotypes due to device state can be accounted for, if the sequence of instructions leading up to the execution of a given instruction is known. For example, the alternative EM morphology of the signal corresponding to an *ldi* instruction is vast. However, the EM morphology range that an *ldi* can take given a sequence of 100 prior instructions is drastically more limited. Therefore, the library for the proposed framework can follow a format as in equation 4.

$$Library = \left[(I_{n-q}, ..., I_{n-2}, I_{n-1}, I_n) \quad S_n^{(I_{n-q},...,I_{n-2},I_{n-1},I_n)} \right] \quad (4)$$

where q is the number of instructions in each sequence, I_n is an ASM instruction supported by the target device, and S_n is a set of EM samples.

However, in real-life conditions, it is not realistic to locate and then harvest the signal corresponding to sequences of

100 instructions from alternative sources (programs). Therefore, we limit ourselves to a more feasible length, i.e., three to five instruction sequences.

4.2 GAN Architecture

The framework capitalizes on the generative capabilities of GANs to convert a series of ASM instructions to high-fidelity synthetic EM signals. ASM instructions are utilized, as opposed to high-level programming languages such as C code. ASM is the most basic form that can be obtained while also having a one-to-one correspondence between each of its instructions and the produced EMs when executing the given instructions. Consequently, using ASM makes the process scalable, and potentially usable for any programming framework e.g., C, Structured Text, Ladder Logic, etc. Moreover, internally, the GAN relies on the principles of CGANs, WGAN-GP, ResGAN, and *tokenization* to tackle practical challenges such as the variable length of programs.

The tokenizer is used to vectorize programs, converting ASM instructions into sequences of integers, as shown in 1 in Figure 6. Each integer corresponds to an index of a token in the tokenizer dictionary. Through tokenization, a given program comprised of several instructions is converted into an equal number of tokens. These tokens are provided as additional input to both the generator and discriminator according to the specifications of CGANs.

The CGAN is adopted mainly as a means of avoiding the complexities of requiring a separate GAN mode for every possible program, as traditional GANs only account for one type/class of output. The framework uses the concept of CGANs to create a single GAN model to generate the EM of *multiple* possible sequences of ASM instructions by providing the tokens to the generator and discriminator as labeled input indicated by 2 in Figure 6. Furthermore, to account for the device state when executing a given instruction, the generator and discriminator models allow for any set sequence size to be used as input. This process is accomplished by passing a number of tokens corresponding to each instruction to the generator and discriminator in the same order seen in the binary ASM code.

The generator and discriminator are also modified to use inner NN layers that are similar to ResGAN, as shown in step 3 in Figure 6. As the generator has to take into account a vast amount of different sequences, the residual learning inside each of these blocks helps avoid the challenge of over-generalization of the EM signals. Consequently, the produced output is more indicative of the input (ASM instruction sequences), which is further improved using WGAN-GP.

WGAN-GP is used to improve the convergence and learning process of the overall GAN model, as illustrated in step 4 in Figure 6. WGAN-GP is mainly required due to the complexity of the model having to generate accurate EM signals for multiple possible sequences for EM-based anomaly

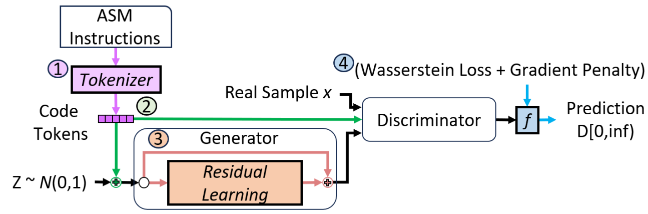

Figure 6. Proposed GAN workflow. Numbers one through four indicate the applied tokenizer, CGANs, ResGAN, and WGAN-GP respectively.

detection, a task that requires high-fidelity in its training samples. As such, the use of Wasserstein loss with gradient penalty provides more information to adjust the weights of the overall GAN model than the traditional loss algorithm. Consequently, the fidelity of the generated signal significantly improves.

In more detail, the structure of the generative network is as follows. Due to ResNet originally being made for images and thus having convolutional layers, the input data must be reshaped to include a third axis. Next, a 1D convolutional layer is applied, effectively replicating the original input among the newly created third axes. The resulting 3-dimensional array is provided as input to a ResNet layer and its inner convolution layers. Afterward, the resulting output is reshaped to a 1D array using global average pooling, and dropout is performed to reduce overfitting complications and improve output range. Finally, several fully connected (dense) layers are applied with the final producing a set-length EM signal. The signal is of set length due to ANNs requiring a specifically identified input and output length.

The discriminator model task is simpler. Its job is to discern between the real and the fake given a pair (signal and token). As such, the discriminator can obtain accurate results given only a few fully connected layers and doesn't require ResNet. The entire GAN model used during experimentation can be seen in Figure 4. The reader should note that the framework can be adapted to use potentially any GAN type/architecture.

4.3 GAN Training

The following outlines the process for training the GAN model. First, the tokens for the matching and prior instructions are obtained via the tokenizer. Second, in a similar fashion to CGANs, random noise is sampled from the latent space and paired with the token from the previous step. The pair is then passed through the generator, which is similar to the one seen in ResGANs, to produce a corresponding synthetic signal. The produced fake signal and samples of real signals from the matching instruction are then passed to the discriminator with the corresponding tokens identifying the instruction sequence in accordance with CGANs. The discriminator then evaluates how well each pair (signal and tokens) correlates. Wasserstein loss is then estimated and is used to update the weights of the discriminator. The new weight is calculated by adding the gradient penalty to the found Wasstersien loss. Finally, the loss is calculated for the generator's ability to fool the discriminator. As such, the generator's weights are updated via the Wasstersien loss function segment pertaining to the fake signals, i.e., $-mean(s_f)$, as it can only control the output of the synthetic signals. This process is repeated for all instruction/EM pairs contained in the library. The entire training algorithm is provided in Algorithm 1.

Algorithm 1 GAN Training algorithm (with step size α)

function GRADIENT_PENALTY:(initial discriminator parameters w, Discriminator output given fake data $\hat{x}$)
    return $(||\nabla_{\hat{x}} D_w(\hat{x})||_2 - 1)^2$
end function
function TRAIN:(minibatch signals s, matching code instructions i, index of matching instruction n, the previous number of training batch steps S, gradient penalty coefficient λ):
    for c = 1 **to** S **do**
        $t \leftarrow \phi([i_{n-1}, i_n])$
        $z \sim N(0,1)^Z$
        $\hat{s} \leftarrow G(z,t)$
        $s_r \leftarrow D(s,t)$
        $s_f \leftarrow D(\hat{s},t)$
        $W_{Loss} \leftarrow -mean(s_f) + mean(s_r)$
        $gp \leftarrow GRADIENT\_PENALTY(w_n, \hat{x})$
        $L_D \leftarrow W\_Loss + \lambda * gp$
        $D \leftarrow D - \alpha \partial L_D / \partial D$
        $L_G \leftarrow -mean(s_f)$
        $G \leftarrow G - \alpha \partial L_G / \partial G$
    end for
end function

4.4 Detection Model

The proposed framework allows for any anomaly detection model to be used for EM-based detection purposes. The only limitation is that the model must be a semi-supervised approach, as opposed to a supervised one. A semi-supervised approach is necessary as nearly infinite alterations to the base code can be made. As such, traditional supervised approaches that require examples and labels for all possible situations are impossible to obtain for EM-based anomaly detection. Fortunately, the number of possible variations under normal operation is finite. Consequently, it is feasible to obtain the range of normalcy (baseline). The process of EM-based anomaly detection can be performed using either shallow ML or DL approach. The major benefit of using shallow ML techniques is that they often obtain accurate results with relatively few samples in comparison to DL. However, ML oftentimes needs prepossessing such as feature engineering and denoising. In comparison, DL methods can outperform ML approaches as they inherently perform such tasks. In this work, we stress that one of the main difficulties of the EM-based anomaly detection process is to harvest high-quality EM signals in volumes to define the baseline of normalcy. The following describes two potential approaches for EM-based anomaly detection.

4.4.1 Semi-supervised AE.

The following details a semi-supervised AE approach for EM-based anomaly detection. In summation, the method uses the *reconstruction error* obtained by comparing the input to the output to identify anomalies. The assumption is that input signals that are similar to the original training samples will yield less error than those that are different.

Training: During training, the AE learns the feature relationships in the training set X. Once again, our training set contains only normal instances of signals. In order to train the AE model to learn the identity function $g(f(x)) = \hat{x}$, the model goes through several epochs (training batches) using standard feed-forward and backpropagation learning. Afterward, the mean square error (mse) is calculated for all x and the resulting $\hat{x}$ in X, as seen in Formula 5:

$$mse(x, \hat{x}) = \frac{1}{n} \sum_{p=1}^{n} (x_p - \hat{x}_p)^2 \tag{5}$$

where n is the number of observations. Finally, a threshold λ is set at or between mse^{min} and mse^{max}. Once again we test all possible λ and report on the best found.

Deployment: Once the autoencoder model is trained and a threshold is derived, a new query signal x_q is obtained during run-time. Signal x_q is passed through the autoencoder to obtain the derived signal representation $\hat{x}_q$. Afterward, the *mse* is calculated for x_q and $\hat{x}_q$. The signal x_q is flagged as normal if the resulting *mse* is below the derived threshold λ, or anomalous otherwise.

5 Experimental Setup

The following specifies the devices, programs, GAN model, and anomaly detection model employed during experimentation.

5.1 Testbed

The capturing process to obtain all the EM signals for our experiments was done on an Arduino Mega device and the overall setup is provided in subfigure *A* of Figure 7. In specifics, the device contains an 8-bit *ATmega2560* AVR microcontroller unit (MCU). This particular MCU is commonly favored for real-time control applications and research. During the execution of a given program, the device emits EMs. These EM emissions are captured using an EMRSS RF Explorer H-Loop *EM* probe positioned directly above the CPU. As the emanation's amplitude is minute, a Behive 150A EMC probe amplifier was used to boost the signal. We relied on a PicoScope 3203D as the oscilloscope of choice. It should be noted that minor displacements of the probe may yield drastically different signals in terms of amplitude and other signal artifacts as those shown in Subfigure *B* of Figure 7.

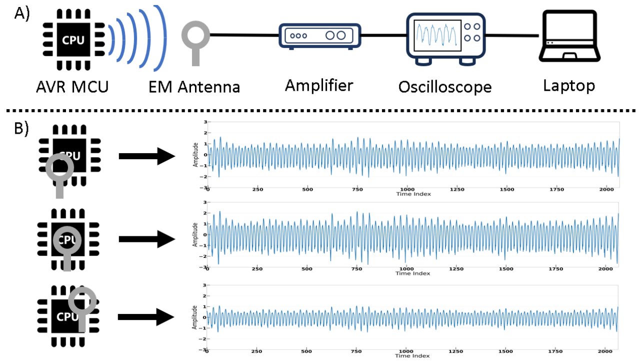

Figure 7. A) Testbed setup. B) Effect of probe positioning.

This is a major practical issue as one cannot expect that the placement of the probe will remain *exactly* the same even after the deployment of the device on the field. This observation has been one of the most important motivations behind this work.

5.2 Programs

For our experiments, a custom program we call *Alpha* for simplicity is taken as our benign case. In specifics, *Alpha* is a program containing 10 ASM instructions that constitute several common sequences of compiled code (ASM) found in the top 20 most-star reviews on GitHub for an *ATmega2560* device. Furthermore, we evaluate our approach under the hardest detection condition (i.e., the addition of one instruction, which in this context is considered a malicious injection). In more detail, the anomalous case during testing contains a single *sub* instruction that is injected into the middle of the binary code of program *Alpha*. The assembly code for *Alpha* is provided in Listing 1.

```
setup:
  sbi ddrb, 6  ; set pb6 as output (our sync artifact)
loop:
  sbi   pinb, 6
  ; Twelve NOPs
  ;==================== Alpha ====================
  ori R16, 200; Logical OR with Immediate.
  sts $FF30, R16; Store Direct to Data Space.
  lds r2, $FF30; Load Direct from Data Space.
  ori R17, 150; Logical OR with Immediate.
  mov r1, r2; Copy Register.
  sub R16, R17; malicious injection
  andi r17, 100; Logical AND with Immediate.
  mov r3, r17; Copy Register.
  ori r18, 77; Logical OR with Immediate.
  mov r4, r18; Copy Register.
  andi r18, 221; Logical AND with Immediate.
  ;===============================================
  ; Twelve NOPs
  ; Clear all registers and flags
  rjmp loop
```

Listing 1. Assembly instructions of program *Alpha*.

5.3 Library

To train our GAN model, ideally an exhaustive library of all possible sizeable sequences or even more (3, 4-instruction, etc.) should exist. However, since for our experiments, the target is only a single program (i.e., the *Alpha*) and to minimize the resources needed to complete the experiment, we developed a stripped-down version of the library containing only the sequences seen in *Alpha*. For this reason, an additional five programs are created strictly for harvesting EM signal subsequences that will be used in the library. Each of the five programs contained roughly 60 ASM instructions. Furthermore, although the code for the five programs overall is different, they collectively contain all sequences of instructions seen in *Alpha*.

5.4 Preprocessing

As EMs are amplitude modulated, the peak values are the main features that indicate the executed instruction at any given time. As such, we chose to focus both the GAN and anomaly detection model on said features (peak amplitude of the EM signal per cycle), effectively reducing the computational requirements for our experiments while simultaneously improving the results. More specifically, for our experiments, the GM is focused on generating just the EM peak amplitudes, and the anomaly detection method is accomplished by evaluating the similarity of signals by comparing only the peaks.

6 Experimental Evaluation

The following section details the results of our experiments aimed at quantifying the effectiveness of our proposed framework for detecting minimal alterations to a program's base ASM code.

6.1 Datasets

For our experiments, we capture 3000 samples of the program *Alpha* that will act as our benign case. Additionally, we capture 1000 samples of the anomalous case which contains the *sub* injection as shown in Section 5.2. The anomalous case's EMs contain minuscule differences in the morphology of the signal when compared to the benign case, as shown in Figure 8. Consequently, the anomalous case is difficult to correctly identify.

Furthermore, to understand the effects of real-life noise when performing EM-based anomaly detection, various levels of White Gaussian Noise (WGN) are artificially added to the captured signals for both benign and anomalous cases to create several datasets. Various levels of WGN are applied to the signals in a way to keep track of the *Signal-to-Noise-Ratio* (SNR) in the signals and emulate variable environmental noise conditions. SNR is a commonly used metric in signal-processing tasks that measures the quality of the signal.

6.2 Generative Accuracy

The accuracy of our proposed framework's ability to generate EM signals close to real captured versions is evaluated by calculating the similarity (i.e., Euclidean distance). Our

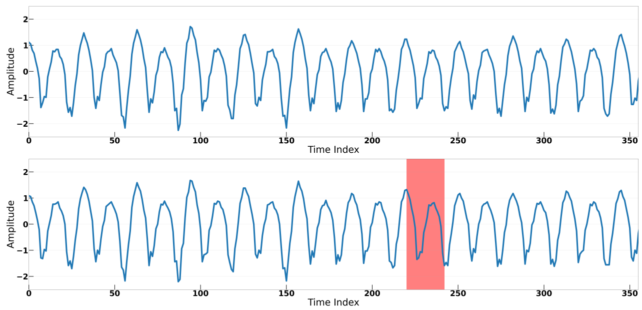

Figure 8. EMs of the Benign (Above) and Anomalous (Below) cases. The injected instruction is highlighted in red.

method for evaluating the similarity to real captured EMs is as follows. First, a signal is obtained from a dataset containing 1500 samples. This signal is then compared to all signals in the base set that contains 1500 real captured signals. This creates a list of distance scores. Next, the 25 most similar (lowest) distance scores are averaged. This repeats for each signal in the first stated dataset, resulting in a range that indicates the similarity to the real EMs. We evaluate the similarity for the following three cases.

Real Signals: We gather the similarity for *Real* captured signals. To get this measurement, we obtain the similarity between the base set and compare it to other real captured signals of the same program. This will serve as the best case to aim for (i.e., perfect similarity).

Synthetic Signals: We obtain the similarity for *Synthetic* signals of said program produced by our GM. This will show our framework's ability to generate high-fidelity signals. This is done by comparing the base set to a dataset of generated signals created using our framework. We provide two results for this case, one where the dataset contains synthetic signals produced by the GM described in Section 4 when trained on a library containing 100% of seen instructions in the given program and a second with only 90%. Both these cases were considered because under real scenarios it may not be possible to obtain EM samples for all possible sequences, especially those of longer lengths (e.g., three or five). Fortunately, [17] indicates that it is reasonable to obtain a library with at least 90% of the seen sequences in most programs, as only a fraction is commonly executed.

Manually Compiling Signals: In this case, a more simple method to generate signals is performed. This method takes real captured samples of instruction's EMs from various programs. Then, for the first ASM instruction seen in a given program's binary code, an EM sample is randomly obtained for the corresponding instruction and manually appended to a collective. This process is repeated for every instruction seen in the binary code of the given program. We then obtain the similarity of the manually compiled signals to the base case. This similarity serves to validate any benefit of using a GM. The distances for all cases are provided in Figure 9 and

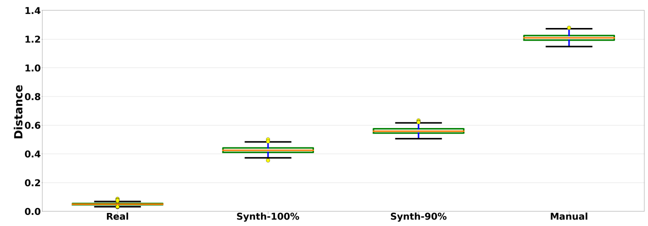

Figure 9. Euclidean distance between real captured EM samples compared against other real and the generated signals.

Table 1. Generative Accuracy statistics.

	Min	Max	AVG	Distance to Real (AVG)
Real	0.030	0.087	0.050	+0.000
Synth-100%	0.354	0.501	0.426	+0.376
Synth-90%	0.506	0.633	0.560	+0.510
Manual	1.148	1.279	1.209	+1.159

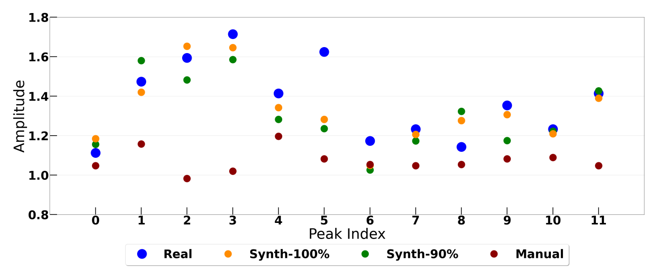

Figure 10. Amplitude peaks between real captured EM samples and generated signals.

Table 1. Additionally, a sample of the EM peak values for all cases is provided in Figure 10.

Results: Looking at Figure 10, while no synthetic peak perfectly overlaps with real, for most cases the amplitude of the real is very close to the synthetic samples. The only exception is the one case at peak amplitude index five.

In Table 1 and Figure 9, the average distances among the *Real* signals are shown to range from 0.030 to 0.087. This is the ideal situation where the EM signals are the same as real captured signals. For the *Synthetically* generated signals, a similarity of 0.354 to 0.501 is achieved when the GM is trained on 100% of seen instructions and 0.506 to 0.633 when trained on 90%. The difference between the two synthetic versions varies slightly, incurring a difference in similarity of only 0.134. This indicates a minimal loss in accuracy when not trained on all instruction sequences. While not perfectly the same as the real captured EMs, the resulting synthetic signals incorporate only minor differences. Finally, the distance of *Manually* created signals is significantly higher. The manual method produces signals whose distance to the real ranges between 1.148 to 1.279.

Table 2. Anomaly Detection Results.

SNR	Metric	K-NN & Real EM Signals. Training Size					AE & Real EM Signals. Training Size					AE & Synthetic EM Signals. Training Size				
		2000	**1500**	**1000**	**500**	**100**	**2000**	**1500**	**1000**	**500**	**100**	**2000**	**1500**	**1000**	**500**	**100**
25	**AUC**	0.996	0.996	0.996	0.994	0.993	0.996	0.995	0.993	0.992	0.978	0.991	0.981	0.979	0.980	0.978
	ACC	0.967	0.969	0.967	0.967	0.966	0.974	0.966	0.960	0.960	0.928	0.955	0.932	0.927	0.928	0.924
	F1	0.967	0.969	0.967	0.967	0.967	0.974	0.966	0.960	0.960	0.930	0.955	0.930	0.929	0.929	0.924
	PREC	0.971	0.970	0.960	0.974	0.962	0.964	0.969	0.962	0.950	0.912	0.959	0.963	0.911	0.914	0.920
	REC	0.966	0.968	0.978	0.960	0.971	0.985	0.964	0.959	0.971	0.948	0.951	0.900	0.947	0.945	0.929
20	**AUC**	0.891	0.892	0.897	0.894	0.874	0.939	0.926	0.927	0.924	0.835	0.944	0.943	0.924	0.921	0.903
	ACC	0.814	0.815	0.821	0.815	0.789	0.874	0.859	0.856	0.853	0.760	0.866	0.866	0.865	0.846	0.825
	F1	0.811	0.815	0.825	0.821	0.800	0.872	0.855	0.853	0.851	0.763	0.863	0.863	0.861	0.845	0.825
	PREC	0.850	0.816	0.836	0.812	0.838	0.886	0.879	0.873	0.862	0.754	0.886	0.886	0.890	0.855	0.825
	REC	0.763	0.813	0.800	0.824	0.785	0.858	0.833	0.833	0.840	0.773	0.841	0.841	0.834	0.835	0.825
15	**AUC**	0.694	0.701	0.694	0.707	0.697	0.786	0.773	0.771	0.767	0.735	0.794	0.793	0.782	0.781	0.777
	ACC	0.653	0.660	0.652	0.660	0.652	0.721	0.711	0.702	0.704	0.677	0.723	0.731	0.711	0.714	0.715
	F1	0.686	0.687	0.683	0.689	0.695	0.714	0.726	0.661	0.690	0.683	0.728	0.718	0.705	0.728	0.721
	PREC	0.647	0.671	0.682	0.653	0.649	0.732	0.689	0.768	0.724	0.670	0.714	0.756	0.721	0.694	0.706
	REC	0.674	0.631	0.575	0.682	0.661	0.697	0.768	0.581	0.660	0.697	0.743	0.683	0.689	0.765	0.738

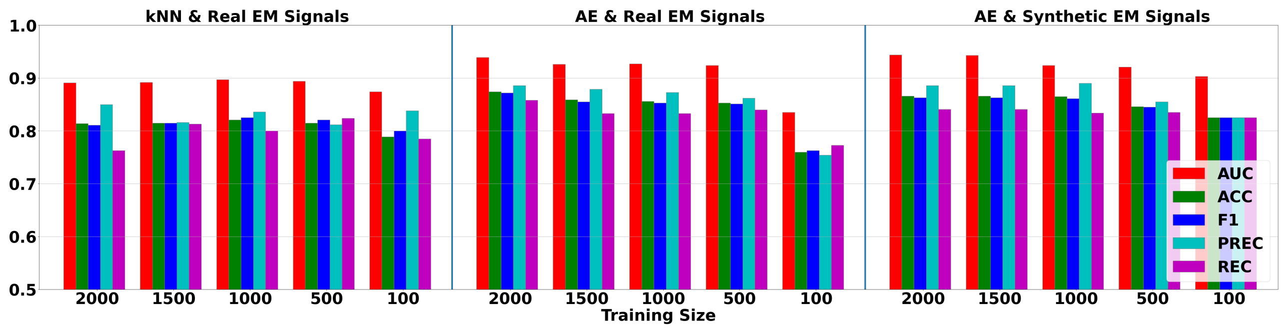

Figure 11. Bar graph for the detection results at 20 SNR.

6.3 Anomaly Detection

In this experiment, we perform the following tests to evaluate the performance of our proposed framework for using a GM for fingerprinting purposes to train an EM-based anomaly detection model. The reader should note that our *goal is to validate our approach of utilizing a GM to address the challenge of fingerprinting, and not to provide a novel EM-based anomaly detection model.* Thus, for this experiment, we evaluate whether training with synthetic signals is a valuable option by comparing the detection results when using our framework to when using the real signal equivalent when training the anomaly detection model. In the first test, we evaluate a semi-supervised k-NN approach training on real captured EMs of the program *Alpha*. This initial test serves as a basis to compare against. The second test uses a semi-supervised AE for anomaly detection instead. Finally, the third test repeats the experiment using AE but is trained on synthetic samples.

For the first two tests, the training set contains various amounts of real captured EM samples of the program *Alpha*. The individual training sizes tested are 2000, 1500, 1000, 500 and 100. For the final experiment, we once again train using the same training sizes, but with synthetically generated signals produced from our GM. For all tests performed, the testing set contains 1000 samples of *Alpha* that are different captures from the ones used in training to serve as the normal case. Furthermore, the testing set is modified by applying WGN to obtain various SNR levels. This is to obtain results that simulate realistic noisy conditions. The training set and testing set have the same SNR levels of noise when using real captured EMs to simulate the traditional methodology of EM-based anomaly detection, i.e., capturing the training set right before deployment to mitigate errors due to uncontrollable phenomena. The various tests are evaluated using the same training and testing sets where appropriate (e.g., in the first two tests, the training and testing sets used for the k-NN model are the same ones used for the Autoencoder model at SNR 15 with 1000 training size).

For the following experiment, we report on the most commonly used metrics for analyzing detection capabilities. Specifically, we provided the area under the curve (AUC) of the receiver operating characteristic (ROC), accuracy (ACC), F1 score, precision (PREC), and recall (REC). The results of the entire experiment are provided in Table 2. In addition, we also provided an ROC comparison at SNR 20 provided in Figure 12.

6.3.1 k-NN & Real EM Signals. For the following test, we identify the accuracy of a rudimentary detection approach commonly used in EM-based anomaly detection. Specifically, we use a semi-supervised k-NN approach, trained on real captured EM signals. For this experiment, the similarity is estimated for the $k = 10$ nearest neighbors and we validate using 3-fold cross-validation. Results of the k-NN test are provided on the left side of Table 2.

Results: Even a low number of signals gives good results in clean environments. At 25 SNR, the results are near perfect, obtaining an AUC of 0.993 when training on 100 EMs. The improvement in results is minuscule with higher training sizes, with a small improvement of 0.003 AUC at with 2000 signals. The results for 25 SNR indicate that k-NN can detect minimal code injections with above 0.95 scores for all metrics when in at least 25 SNR environments. At SNR 15, all metrics drop below 0.9. The AUC achieved is 0.874 at 100 samples and improving only slightly to around 0.89 with more. Finally, at SNR 15, the results fall to 0.697 AUC when training on 100 EMs, and no significant improvement is seen with more samples. As such, SNR 15 is below the limit where we consider our method accurate.

Conclusions: k-NN works well in low noise level environments (25 SNR and above). Additionally, k-NN is able to reach its peak performance with relatively few (100) samples. However, the method suffers at higher noise levels. Consequently, either a different model that can handle noise such as AE, or a noise reduction prepossessing method is required to accurately identify anomalies below 25 SNR.

6.3.2 Autoencoder & Real EM Signals. In the second test, we evaluate using the semi-supervised autoencoder approach for anomaly detection described in Section 4.4.1. AE is an NN framework that could improve results as it automatically performs dimensionality reduction on the features. Therefore, in theory, by training a model to reduce key features, the negative effect that noise to hide those features is removed. Furthermore, DL models have been shown to outperform ML approaches in various tasks so long as enough training samples are provided. As such, the AE could better detect anomalies at lower SNR than k-NN. Overall, this test is performed to improve results and identify any inherent benefit of using DL. We evaluate using 3-fold cross-validation. Results are provided in the middle of Table 2.

Results: The results from this experiment show improvement with higher training sizes. Near-perfect results are seen at SNR 25, with results above 0.99 when training on 500 and above samples. Looking at SNR 20, the AUC at 100 training size is 0.834. This metric improves at nearly every higher training size, even up to 2000 with an AUC of 0.939. This trend is also seen at SNR 15. However, while the AE method obtains results above 0.9 at SNR 20, it still struggles at SNR 15, achieving an AUC of 0.786 (2000 training samples).

Conclusions: Both methods at SNR 25 obtain similar results, with k-NN achieving its best scores at 0.996 AUC, 0.967 ACC and F1, 0.971 PREC and 0.966 REC, and AE with 0.996 AUC and 0.974 ACC and F1, 0.964 PREC and 0.985 REC. Additionally, AE outperforms k-NN at the lower SNRs. Overall, when comparing the best AUC across all training sizes, AE outperforms k-NN by 0.04 AUC at SNR 20 and 0.08 AUC at SNR 15. Another important observation is that AE needs much larger training datasets than k-NN to provide optimal results. A comparison between the AE method and the k-NN method can be seen in Figure 12.

6.3.3 Autoencoder & Synthetic EM Signals. In our final test, our proposed framework is utilized. More specifically, our GM is trained to synthetically generate the normal instances that are used for training the anomaly detection method. Overall, this experiment serves to quantify the performance of our proposed framework. The results of this experiment are provided on the right in Figure 12.

Results: The results obtained using our proposed framework are near-perfect at SNR 25, achieving a high of 0.991 AUC and 0.955 ACC and F1, 0.959 PREC and 0.951 REC. However, the detection performance decreases the lower the SNR, obtaining the best results of 0.944 AUC, 0.866 ACC, 0.863 F1, 0.886 PREC and 0.841 REC at 20 SNR and 0.794 AUC, 0.723 ACC, 0.728 F1, 0.714 PREC and 0.743 REC at SNR 15. Furthermore, the trend of improved performance with higher training sizes is seen, similar to the prior AE experiment using real signals.

Conclusions: In comparison to the previous AE results when utilizing a traditional EM-based anomaly detection methodology with real capture signals, the results are nearly identical. When looking at 2000 training samples, at SNR 25 there is a minimal decrease of AUC -0.005 when using our framework. However, at SNR 20 and SNR 15, the AUC improves by +0.004 and +0.008.

In addition, a comparison can be made with k-NN using the traditional EM-based anomaly detection method and our proposed approach using AE. In Figure 12, the AE model outperforms k-NN when training on 2000 synthetic samples produced by our GM. Therefore, EM-based anomaly detection can improve by using our proposed framework with DL methods such as AEs.

7 Conclusions and Future Directions

In this paper, we provide a novel framework that removes the need for manually capturing EM emanations of digital device components that can be used for fingerprinting purposes in the context of EM-based anomaly detection. Instead, our proposed framework utilizes a GM based on WGANs-GP, CGAN, and ResGAN to synthetically generate EM signals given the binary assembly code and a library. Through experimentation, we provided validation that our framework

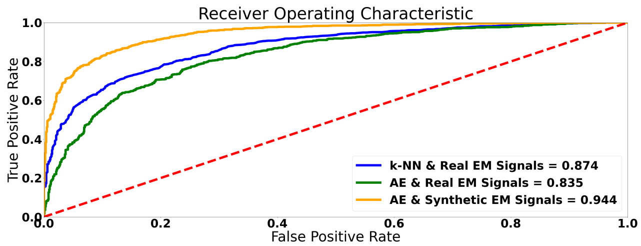

Figure 12. ROCs for anomaly detection experiment at 20 SNR with a training size of 100 when training on real samples and 2000 when training on synthetic samples.

is able to generate synthetic signals that accurately approximate the real captured EM signals for a given program. Thus, these synthetic signals can be utilized to train an EM-based anomaly detection model to the point that it outperforms other models that were trained using real signals.

Future extensions of this work include incorporating *transferability* methods to account for changes regarding the location of the EM-probe, type of EM-probe, or the device model. By incorporating such methods, we aim to remove the need for training multiple GMs, each one specific to a particular setting/setup. Furthermore, we will compare the use of our framework with a wider range of anomaly detection approaches and evaluate using more programs comprised of a wider range of machine instructions.

8 Acknowledgements

Effort performed through Department of Energy under U.S. DOE Idaho Operations Office Contract DE-AC07-05ID14517, as part of Laboratory Directed Research and Development, Resilient Control and Instrumentation Systems (ReCIS) program of Idaho National Laboratory.

References

[1] Martin Arjovsky, Soumith Chintala, and Léon Bottou. 2017. Wasserstein generative adversarial networks. In *International conference on machine learning*. PMLR, 214–223.

[2] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2020. Generative adversarial networks. *Commun. ACM* 63, 11 (2020), 139–144.

[3] Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron C Courville. 2017. Improved Training of Wasserstein GANs. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc.

[4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.

[5] Haider Adnan Khan, Monjur Alam, Alenka Zajic, and Milos Prvulovic. 2018. Detailed tracking of program control flow using analog side-channel signals: a promise for iot malware detection and a threat for many cryptographic implementations. In *Cyber Sensing 2018*, Vol. 10630. International Society for Optics and Photonics, 1063005.

[6] Diederik P Kingma, Max Welling, et al. 2019. An introduction to variational autoencoders. *Foundations and Trends® in Machine Learning* 12, 4 (2019), 307–392.

[7] Constantinos Kolias, Daniel Barbará, Craig Rieger, and Jacob Ulrich. 2020. Em fingerprints: Towards identifying unauthorized hardware substitutions in the supply chain jungle. In *2020 IEEE Security and Privacy Workshops (SPW)*. IEEE, 144–151.

[8] Amit Kumar, Cody Scarborough, Ali Yilmaz, and Michael Orshansky. 2017. Efficient simulation of EM side-channel attack resilience. In *2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 123–130.

[9] Songxiang Liu, Dan Su, and Dong Yu. 2022. Diffgan-tts: High-fidelity and efficient text-to-speech with denoising diffusion gans. *arXiv preprint arXiv:2201.11972* (2022).

[10] Yannan Liu, Lingxiao Wei, Zhe Zhou, Kehuan Zhang, Wenyuan Xu, and Qiang Xu. 2016. On code execution tracking via power side-channel. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*. 1019–1031.

[11] Yongyi Lu, Shangzhe Wu, Yu-Wing Tai, and Chi-Keung Tang. 2018. Image generation from sketch constraint using contextual gan. In *Proceedings of the European conference on computer vision (ECCV)*. 205–220.

[12] Ekaterina Miller. 2022. *Detecting Code Injections in Noisy Environments through EM Signal Analysis and SVD Denoising*. Master's thesis. University of Idaho. https://www.proquest.com/dissertations-theses/detecting-code-injections-noisy-environments/docview/2674042935/se-2

[13] Seun Sangodoyin, Frank T Werner, Baki B Yilmaz, Chia-Lin Cheng, Elvan M Ugurlu, Nader Sehatbakhsh, Milos Prvulović, and Alenka Zajic. 2020. Side-channel propagation measurements and modeling for hardware security in iot devices. *IEEE Transactions on Antennas and Propagation* 69, 6 (2020), 3470–3484.

[14] Nader Sehatbakhsh, Monjur Alam, Alireza Nazari, Alenka Zajic, and Milos Prvulovic. 2018. Syndrome: Spectral analysis for anomaly detection on medical iot and embedded devices. In *2018 IEEE international symposium on hardware oriented security and trust (HOST)*. IEEE, 1–8.

[15] Nader Sehatbakhsh, Baki Berkay Yilmaz, Alenka Zajic, and Milos Prvulovic. 2020. EMSim: A Microarchitecture-Level Simulation Tool for Modeling Electromagnetic Side-Channel Signals. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 71–85. https://doi.org/10.1109/HPCA47549.2020.00016

[16] Kurt Vedros, Georgios Michail Makrakis, Constantinos Kolias, Min Xian, Daniel Barbara, and Craig Rieger. 2021. On the Limits of EM Based Detection of Control Logic Injection Attacks In Noisy Environments. In *2021 Resilience Week (RWS)*. IEEE, 1–9.

[17] Kurt A Vedros, Constantinos Kolias, and Robert C Ivans. 2023. Do Programs Dream of Electromagnetic Signals? Towards GAN-based Code-to-Signal Synthesis. In *MILCOM 2023-2023 IEEE Military Communications Conference (MILCOM)*. IEEE, 716–721.

[18] Kurt A Vedros, Georgios Michail Makrakis, Constantinos Kolias, Robert C Ivans, and Craig Rieger. 2023. Towards Scalable EM-based Anomaly Detection For Embedded Devices Through Synthetic Fingerprinting. *arXiv preprint arXiv:2302.02324* (2023).

[19] Meng Wang, Huafeng Li, and Fang Li. 2017. Generative adversarial network based on resnet for conditional image restoration. *arXiv preprint arXiv:1707.04881* (2017).

[20] Shijia Wei, Aydin Aysu, Michael Orshansky, Andreas Gerstlauer, and Mohit Tiwari. 2019. Using power-anomalies to counter evasive micro-architectural attacks in embedded systems. In *2019 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, 111–120.

Received 13 March 2024; accepted 3 May 2024